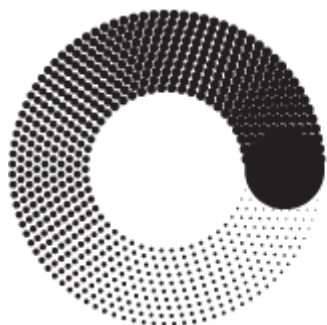


Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования



**МОСКОВСКИЙ
ПОЛИТЕХ**

ОТЧЕТ

по Лабораторной работе 4

по дисциплине: «Технологии компьютерной графики»

Выполнил:	Назаров Т. Д.
Студент Группы:	231-321
Проверили:	Пухова Е.А. Верещагин В.Ю

Москва 2026

1. Цель работы

Изучить методы двумерной интерполяции для масштабирования изображений - реализовать собственные алгоритмы масштабирования (ближайший сосед и билинейная интерполяция), добавить инструмент изменения размера изображения в существующее приложение.

2. Постановка задачи

2.1 Реализация интерполяции

Необходимо написать собственную реализацию двух методов двумерной интерполяции - метод ближайшего соседа и билинейную интерполяцию. Оба метода должны работать с изображением и позволять изменять его размер до произвольной ширины и высоты. По умолчанию используется билинейная интерполяция. Программный интерфейс должен быть спроектирован так, чтобы в будущем можно было легко добавить новые методы интерполяции.

2.2 Отображение на главном экране

В информационную панель необходимо добавить ползунок (range) с диапазоном масштаба от 12% до 300%. При изменении значения масштаба изображение должно перерисовываться в центре canvas с учетом выбранного масштаба. При первоначальной загрузке изображение должно целиком помещаться в главный canvas с отступами минимум 50 пикселей, при этом масштаб не должен выходить за диапазон от 12% до 300%.

2.3 Инструмент изменения масштаба

Требуется создать инструмент изменения масштаба изображения с использованием модального окна. В окне должна отображаться информация об общем количестве пикселей до и после изменения размера (в мегапикселях). Выпадающий список позволяет выбрать единицы измерения - проценты или пиксели. Два поля ввода для указания высоты и ширины в выбранных единицах. Чекбокс для сохранения пропорций исходного изображения при изменении размера. Выпадающий список для выбора алгоритма интерполяции (ближайший сосед или билинейная). Тултип с кратким описанием выбранного алгоритма и его преимуществ.

Необходимо реализовать валидацию вводимых данных - значения не должны быть меньше 1, процентный масштаб должен быть в диапазоне от 12% до 300%, размер в пикселях не должен превышать 8000 пикселей. После подтверждения действия модальное окно закрывается, создается новое изображение с заданным размером, и оно отображается на canvas.

3. Теоретическая часть

3.1 Метод ближайшего соседа

Метод ближайшего соседа - это простейший алгоритм масштабирования. Для каждого пикселя выходного изображения вычисляется соответствующий пиксель исходного изображения путем округления координат, и его значение копируется в выходной массив. Формула преобразования: $srcX = \text{round}(x * (\text{srcWidth} / \text{dstWidth}))$, $srcY = \text{round}(y * (\text{srcHeight} / \text{dstHeight}))$. Преимущества метода - максимальная скорость выполнения. Недостатки - появление пикселизации при увеличении и ступенчатые диагонали.

3.2 Билинейная интерполяция

Билинейная интерполяция - это более качественный метод масштабирования, который использует четыре соседних пикселя для вычисления взвешенного среднего. Сначала выполняется линейная интерполяция по горизонтали между левой и правой парой пикселей, затем - по вертикали между полученными значениями. Преимущества - гладкое изображение без пикселизации. Недостатки - меньшая скорость выполнения и небольшое размытие.

3.3 Формула масштабирования с сохранением пропорций

При масштабировании с сохранением пропорций вычисляется коэффициент масштаба как минимальное отношение отображаемой области к размерам изображения. Формула: $scale = \min(\text{containerWidth} / \text{imageWidth}, \text{containerHeight} / \text{imageHeight})$. Затем новые размеры вычисляются как $\text{imageWidth} * scale$ и $\text{imageHeight} * scale$.

4. Реализация

4.1 Архитектура модуля ImageScaler

```
96 // nearestNeighbor
97 class ImageScaler {
98     static nearestNeighbor(srcData, srcW, srcH, dstW, dstH) {
99         const dstData = new Uint8ClampedArray(dstW * dstH * 4);
100         const xRatio = srcW / dstW;
101         const yRatio = srcH / dstH;
102         for (let y = 0; y < dstH; y++) {
103             const srcY = Math.min(Math.floor(y * yRatio), srcH - 1);
104             for (let x = 0; x < dstW; x++) {
105                 const srcX = Math.min(Math.floor(x * xRatio), srcW - 1);
106                 const srcIdx = (srcY * srcW + srcX) * 4;
107                 const dstIdx = (y * dstW + x) * 4;
108                 dstData[dstIdx] = srcData[srcIdx];
109                 dstData[dstIdx + 1] = srcData[srcIdx + 1];
110                 dstData[dstIdx + 2] = srcData[srcIdx + 2];
111                 dstData[dstIdx + 3] = srcData[srcIdx + 3];
112             }
113         }
114         return dstData;
115     }
116 }
```

```

117     static bilinear(srcData, srcW, srcH, dstW, dstH) {
118         const dstData = new Uint8ClampedArray(dstW * dstH * 4);
119         const xRatio = srcW / dstW;
120         const yRatio = srcH / dstH;
121         for (let y = 0; y < dstH; y++) {
122             const srcY = y * yRatio;
123             const y1 = Math.floor(srcY);
124             const y2 = Math.min(y1 + 1, srcH - 1);
125             const dy = srcY - y1;
126             for (let x = 0; x < dstW; x++) {
127                 const srcX = x * xRatio;
128                 const x1 = Math.floor(srcX);
129                 const x2 = Math.min(x1 + 1, srcW - 1);
130                 const dx = srcX - x1;
131                 const idx11 = (y1 * srcW + x1) * 4;
132                 const idx21 = (y1 * srcW + x2) * 4;
133                 const idx12 = (y2 * srcW + x1) * 4;
134                 const idx22 = (y2 * srcW + x2) * 4;
135                 for (let c = 0; c < 4; c++) {
136                     const v11 = srcData[idx11 + c];
137                     const v21 = srcData[idx21 + c];
138                     const v12 = srcData[idx12 + c];
139                     const v22 = srcData[idx22 + c];
140                     const v1 = v11 * (1 - dx) + v21 * dx;
141                     const v2 = v12 * (1 - dx) + v22 * dx;
142                     const val = v1 * (1 - dy) + v2 * dy;
143                     const dstIdx = (y * dstW + x) * 4 + c;
144                     dstData[dstIdx] = Math.min(255, Math.max(0, Math.round(val)));
145                 }
146             }
147         }
148         return dstData;
149     }
150 }

151 static scale(imageData, newWidth, newHeight, method = 'bilinear') {
152     const srcData = imageData.data;
153     const srcW = imageData.width;
154     const srcH = imageData.height;
155     let dstData;
156     if (method === 'nearest') {
157         dstData = this.nearestNeighbor(srcData, srcW, srcH, newWidth, newHeight);
158     } else {
159         dstData = this.bilinear(srcData, srcW, srcH, newWidth, newHeight);
160     }
161     const newImageData = new ImageData(newWidth, newHeight);
162     newImageData.data.set(dstData);
163     return newImageData;
164 }
165 }

```

4.2 Масштабирование с вписыванием в canvas

```

551  fitCanvas() {
552      if (!this.currentImageData) return;
553      const container = document.querySelector('.canvas-container');
554      if (!container) return;
555      const containerWidth = container.clientWidth - 40;
556      const containerHeight = 500;
557      const imgWidth = this.currentImageData.width;
558      const imgHeight = this.currentImageData.height;
559      const scale = Math.min(containerWidth / imgWidth, containerHeight / imgHeight, 1);
560      const scalePercent = Math.round(scale * 100);
561      const scaleRange = document.getElementById('scaleRange');
562      const scalePercentSpan = document.getElementById('scalePercent');
563      if (scaleRange && scalePercentSpan && this.currentScale !== scalePercent) {
564          this.currentScale = scalePercent;
565          scaleRange.value = scalePercent;
566          scalePercentSpan.textContent = `${scalePercent}%`;
567          if (this.labScale) this.labScale.textContent = `${scalePercent}%`;
568      }
569      this.canvas.style.width = `${imgWidth * scale}px`;
570      this.canvas.style.height = `${imgHeight * scale}px`;
571      this.canvas.width = imgWidth;
572      this.canvas.height = imgHeight;
573      this.ctx.putImageData(this.currentImageData, 0, 0);
574  }

```

4.3 Валидация данных в модальном окне

```

1093  applyScale() {
1094      if (!this.app.originalImageData) return;
1095      let newW = parseInt(this.scaleWidth.value);
1096      let newH = parseInt(this.scaleHeight.value);
1097      if (isNaN(newW) || isNaN(newH) || newW < 1 || newH < 1) {
1098          this.app.showToast('Введите корректные размеры!', 'error');
1099          return;
1100      }
1101      if (this.scaleUnit.value === 'percent') {
1102          newW = Math.round(this.app.originalImageData.width * newW / 100);
1103          newH = Math.round(this.app.originalImageData.height * newH / 100);
1104      }
1105      newW = Math.min(8000, Math.max(1, newW));
1106      newH = Math.min(8000, Math.max(1, newH));
1107      const method = this.interpolationMethod.value;
1108      this.app.showToast(`Масштабирование: ${this.app.originalImageData.width}x${this.app.originalImageData.height} → ${newW}x${newH}`);
1109      const newImageData = ImageScaler.scale(this.app.originalImageData, newW, newH, method);
1110      this.app.originalImageData = newImageData;
1111      this.app.currentImageData = this.app.cloneImageData(newImageData);
1112      this.currentScale = 100;
1113      this.scaleRange.value = 100;
1114      this.scalePercent.textContent = '100%';
1115      if (this.app.labScale) this.app.labScale.textContent = '100%';
1116      this.app.generateChannelPreviews();
1117      this.app.applyChannels();
1118      this.app.renderCanvas();
1119      this.scaleDialog.close();
1120      const labWidth = document.getElementById('labWidth');
1121      const labHeight = document.getElementById('labHeight');
1122      if (labWidth) labWidth.textContent = newW;
1123      if (labHeight) labHeight.textContent = newH;
1124      this.app.showToast(`✓ Изображение масштабировано до ${newW}x${newH}`, 'success');
1125  }

```

5. Интерфейс пользователя

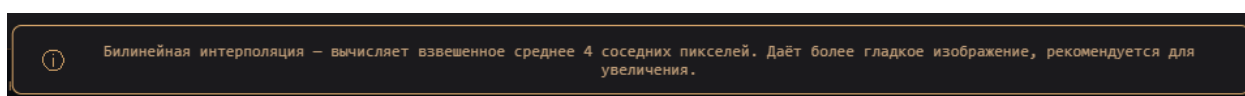
5.1 Панель масштаба

Панель масштаба содержит ползунок с диапазоном от 12% до 300%, отображение текущего значения в процентах и кнопку «НАСТРОЙКИ» для открытия расширенного модального окна.



5.2 Модальное окно масштабирования

В модальном окне отображается информация о количестве мегапикселей до и после изменения. Выпадающий список позволяет выбрать единицы измерения - проценты или пиксели. Два поля ввода для ширины и высоты. Чекбокс для сохранения пропорций. Выпадающий список для выбора алгоритма интерполяции с тултипом. Кнопки ОТМЕНА и ПРИМЕНИТЬ.

A screenshot of a modal window titled 'МАСШТАБИРОВАНИЕ ИЗОБРАЖЕНИЯ' (Image Scaling). The window contains the following elements: a header with the title and a close button (X); a section showing 'ДО: 2.07 МП' (Before: 2.07 MP) and 'ПОСЛЕ: 2.07 МП' (After: 2.07 MP); a 'ЕДИНИЦЫ:' (Units) dropdown menu currently set to 'ПРОЦЕНТЫ (%)' (Percent (%)); input fields for 'ШИРИНА: 100 %' (Width: 100 %) and 'ВЫСОТА: 100 %' (Height: 100 %); a checked checkbox labeled 'СОХРАНИТЬ ПРОПОРЦИИ' (Save proportions); an 'АЛГОРИТМ:' (Algorithm) dropdown menu set to 'БИЛИНЕЙНАЯ ИНТЕРПОЛЯЦИЯ' (Bilinear interpolation) with a tooltip icon; and two buttons at the bottom: 'ОТМЕНА' (Cancel) and 'ПРИМЕНИТЬ' (Apply).

5.3 Статус-бар

В статус-баре добавлен новый пункт «МАСШТАБ», который отображает текущее значение масштаба в процентах.

6. Заключение

В ходе выполнения лабораторной работы реализованы два метода интерполяции для масштабирования изображений - ближайший сосед и билинейная интерполяция. Добавлен ползунок масштаба от 12% до 300% с мгновенным обновлением изображения. Реализовано модальное окно с расширенными настройками масштабирования, включая выбор единиц измерения, сохранение пропорций, выбор алгоритма интерполяции с тултипом и валидацию вводимых данных. При первоначальной загрузке изображение автоматически вписывается в canvas с отступами.